



POLITÉCNICA

Universidad Politécnica de Madrid

Asignatura: Bases de Datos II

Práctica: Arkham Dread en Redis

Caché, Búsqueda Avanzada (Redisearch) y Recomendación (Full-text y Vector Search)

Autores

Yixuan Lu Guo
Shengkai Zhu

23 de marzo de 2026

Índice

1. Introducción	2
1.1. Objetivos implementados	2
2. Datos y entorno de trabajo	2
2.1. Dataset	2
2.2. Tecnologías	2
3. Objetivo I: Caché en Redis	3
3.1. Estructura de datos elegida	3
3.2. Carga del dataset	3
3.3. Operaciones básicas de la caché	3
3.4. Validación con ejemplos	3
4. Objetivo II: Búsqueda avanzada con RedisSearch	3
4.1. Diseño del índice	4
4.2. Helper de parseo: <code>_parse_ft_search</code>	4
4.3. II-A: Búsqueda por facciones (AND/OR) — cartas multifacción	4
4.4. II-B: Traits más comunes por facción	5
4.5. II-C: Upgrades con <code>xp>0</code> , excluir <code>mythos</code> y priorizar facción	6
4.6. Cómo verificamos que las búsquedas son correctas	6
5. Objetivo III: Recomendador de cartas	7
5.1. Embeddings y representación vectorial	7
5.2. Índice para recomendación	7
5.3. Alternativa A: Recomendación por metadatos	7
5.4. Alternativa B: Recomendación full-text	8
5.5. Alternativa C: Recomendación semántica (embeddings + KNN)	8
5.6. Prueba comparativa con la carta 1029 (Shotgun)	8
5.6.1. Resultados A: Metadatos	8
5.6.2. Resultados B: Full-text	8
5.6.3. Resultados C: Semántica (embeddings + KNN)	9
6. Discusión y análisis comparativo	9
6.1. ¿Qué aporta cada enfoque?	9
6.2. Coste vs calidad	9
7. Conclusiones	10

1. Introducción

El problema de partida es bastante realista: el portal web del juego de cartas *Arkham Dread*, servido por una base de datos relacional, empieza a sufrir por volumen de consultas. Nos piden diseñar una caché en Redis y, de paso, aprovechar sus capacidades de búsqueda avanzada para implementar funcionalidades que los usuarios llevan tiempo pidiendo.

El dataset tiene **5346 cartas** y, por cada carta, manejamos campos como `name`, `text`, `type_code`, `traits`, `faction_code`, `xp`, etc. Algunos de estos campos son multi-valor y vienen separados por `|`. Por ejemplo, una carta puede pertenecer a dos facciones (`guardian|seeker`) o tener varios traits (`Item|Weapon|Firearm`). Esto condiciona bastante el diseño de los índices.

1.1. Objetivos implementados

- **Objetivo I: Caché en Redis.** Elegir estructura de datos, cargar el CSV y soportar las operaciones básicas: existencia, recuperar, insertar y borrar.
- **Objetivo II: Búsqueda avanzada con RediSearch.**
 - A) Búsqueda de cartas multifacción con modos AND y OR, paginando de 5 en 5.
 - B) Traits más comunes por facción mediante agregación, paginando de 15 en 15.
 - C) Búsqueda de upgrades: cartas con `xp>0`, excluyendo `mythos`, con filtro por `xp_max` y priorizando la facción del jugador.
- **Objetivo III: Recomendador de cartas.** Tres alternativas: A) metadatos (facción + traits), B) full-text con eliminación de stopwords, C) semántica (embeddings + KNN vectorial).

2. Datos y entorno de trabajo

2.1. Dataset

Trabajamos con `cards_final_with_xp.csv`, un fichero con **5346 filas** (una por carta). Las columnas son:

- `code` (identificador único), `name`, `text`
- `type_code`, `pack_code`, `illustrator`, `image_url`
- `traits` (posible multi-valor, separado por `|`)
- `faction_code` (posible multi-valor, separado por `|`)
- `xp` (entero, típicamente 0–5, que refleja el coste de experiencia)

2.2. Tecnologías

- **Redis Stack** en local (`localhost:6379`), lanzado con Docker:
`docker run -rm -name redis-stack -p 6379:6379 -p 8001:8001 redis/redis-stack:latest`
- **RediSearch** para indexación y consultas (`FT.CREATE`, `FT.SEARCH`, `FT.AGGREGATE`).
- **Python:** `redis-py`, `pandas`, `nltk` (para stopwords en full-text) y `sentence-transformers`.
- **Modelo de embeddings:** `all-MiniLM-L6-v2`, que genera vectores de dimensión **384**.

3. Objetivo I: Caché en Redis

3.1. Estructura de datos elegida

Optamos por usar **Redis Hashes**, donde cada carta es un hash con clave `card:<code>`. Las razones son las habituales: acceso en $O(1)$ por clave, posibilidad de leer campos individuales con `HGET` o el hash completo con `HGETALL`, y actualización parcial sin reescribir todo el objeto. Para una caché de cartas donde cada registro tiene unos 10 campos, es la opción más directa y la que mejor se adapta.

3.2. Carga del dataset

Leemos el CSV con `pandas` y recorremos cada fila insertando un hash en Redis. La función `cargar_cartas_en_redis` hace un `r.hset(key, mapping=data)` por cada carta, donde `data` es el diccionario que sale de `serie.to_dict()` tras extraer el `code` para formar la clave.

```
def cargar_cartas_en_redis(df: pd.DataFrame) -> None:
    for _, serie in df.iterrows():
        data = serie.to_dict()
        code = data.pop("code", None)
        if code:
            key = f"card:{code}"
            r.hset(key, mapping=data)
```

Es un enfoque sencillo. En un entorno real con miles de cartas convendría usar pipelines de Redis para reducir el número de round-trips, pero para el prototipo esto funciona sin problemas.

3.3. Operaciones básicas de la caché

Implementamos las cuatro operaciones que pide el enunciado, cada una en su propia función:

1. **Comprobar existencia** (`carta_en_cache`): usa `r.exists(key)` y devuelve un booleano.
2. **Recuperar datos** (`recuperar_carta`): usa `r.hgetall(key)` y decodifica las claves y valores de bytes a strings UTF-8, devolviendo un diccionario o `None` si no existe.
3. **Insertar carta** (`meter_carta_en_cache`): comprueba primero que no exista ya (para no sobrescribir) y luego hace `r.hset`.
4. **Eliminar carta** (`eliminar_carta_de_cache`): comprueba existencia y usa `r.delete(key)`.

3.4. Validación con ejemplos

Comprobamos el flujo completo: 01001 no estaba en caché (`False`), 06095 sí (`True`). Después insertamos una carta ficticia 01001 (`Test Card`), verificamos que se recuperaba correctamente con todos sus campos, y finalmente la eliminamos y confirmamos que `recuperar_carta` volvía `None`.

4. Objetivo II: Búsqueda avanzada con RedisSearch

Aquí ya no basta con buscar por clave. Necesitamos consultas sobre campos concretos (facciones, traits, rangos de xp), con paginación y, en algunos casos, ordenación con prioridad.

4.1. Diseño del índice

Creamos un índice `cards-idx` sobre los hashes con prefijo `card:`. Las decisiones de diseño fueron:

- `faction_code` como **TAG** con **SEPARATOR** `"|"`: imprescindible para soportar cartas multifacción. Al usar TAG, RedisSearch entiende que `guardian|seeker` son dos tags distintos y permite buscar por cada uno.
- `traits` como **TAG** con **SEPARATOR** `"|"`: misma lógica, para cartas con varios traits.
- `xp` como **NUMERIC** y **SORTABLE**: para poder filtrar por rangos (ej.: `xp>0`, `xp<=5`) y ordenar si hace falta.
- `name` y `text` como **TEXT**: para habilitar búsquedas full-text que explotamos sobre todo en el Objetivo III.
- También indexamos `code` como TAG y `illustrator`, `image_url` como TEXT, más por utilidad en los RETURN que por búsqueda.

```
r.execute_command(
    "FT.CREATE", index_name, "ON", "HASH",
    "PREFIX", "1", prefix, "SCHEMA",
    "code", "TAG",
    "name", "TEXT",
    "text", "TEXT",
    "type_code", "TAG",
    "pack_code", "TAG",
    "faction_code", "TAG", "SEPARATOR", "|",
    "traits", "TAG", "SEPARATOR", "|",
    "xp", "NUMERIC", "SORTABLE",
    "illustrator", "TEXT",
    "image_url", "TEXT"
)
```

4.2. Helper de parseo: `_parse_ft_search`

Antes de entrar en las búsquedas conviene mencionar que implementamos una función auxiliar `_parse_ft_search` que transforma la respuesta cruda de `FT.SEARCH` (formato RESP2: `[total, key1, [field, value, ...], ...]`) en un par `(total, lista_de_dicts)` más manejable. Un detalle que nos costó un rato: si el hash no tiene un campo `code` explícito o no lo pedimos en `RETURN`, lo extraemos del `docid` (la parte después de `card:`). Esto evita que nos salgan resultados con el código vacío.

4.3. II-A: Búsqueda por facciones (AND/OR) — cartas multifacción

Un punto que diferencia nuestra implementación del planteamiento más simple: el enunciado habla de cartas multifacción (cartas especiales con más de una facción), así que nuestra función `search_by_factions` busca exclusivamente cartas que tengan **estrictamente más de una facción**. Se excluyen `neutral` y `mythos` del conjunto válido de facciones para multifacción (tal como dice el enunciado: las cartas multifacción nunca contienen `neutral` ni `mythos`).

La lógica es:

- **Modo AND**: si el usuario pide, por ejemplo, `[guardian, seeker]`, construimos la query `@faction_code:{guardian} @faction_code:{seeker}`, que obliga a que ambas facciones estén presentes. Si solo pide una facción, exigimos que tenga esa y **al menos otra** de las válidas, para garantizar que sea multifacción.

- **Modo OR:** para cada facción pedida, creamos un bloque que exige esa facción **y además otra válida**. Luego unimos todos los bloques con OR (`|`). Así nos aseguramos de que todos los resultados sean cartas multifacción, pero basta con que compartan al menos una de las facciones pedidas.

```
# Ejemplo AND con 2 facciones:
query = "@faction_code:{guardian} @faction_code:{seeker}"

# Ejemplo OR (cada bloque fuerza multifaccion):
block = "(@faction_code:{guardian} @faction_code:{seeker|mystic|...})"
query = block1 + " | " + block2
```

Los resultados se paginan de 5 en 5 con `LIMIT offset page_size`, y devolvemos los campos `code`, `name`, `faction_code` y `xp`.

4.4. II-B: Traits más comunes por facción

Aquí la herramienta clave es `FT.AGGREGATE`. La idea: dado que los traits vienen como un string concatenado con `|` (ej.: `Item|Weapon|Firearm`), necesitamos “descomponerlos” para poder agrupar y contar. RediSearch permite hacer esto con `APPLY split(@traits, '|') AS trait`, lo cual expande cada carta en tantas filas como traits tenga. Después agrupamos con `GROUPBY` y contamos con `REDUCE COUNT`.

El pipeline completo de `FT.AGGREGATE` es:

1. **Filtrar** por facción: `@faction_code:{survivor}`
2. **Cargar** el campo traits: `LOAD 1 traits`
3. **Expandir** traits: `APPLY split(@traits, '|') AS trait`
4. **Agrupar y contar:** `GROUPBY 1 @trait REDUCE COUNT 0 AS frecuencia`
5. **Ordenar** por frecuencia descendente: `SORTBY 2 @frecuencia DESC`
6. **Paginar** de 15 en 15: `LIMIT offset 15`

Ejemplo real (survivor). Para la facción `survivor`, en la primera página (15 traits) obtuvimos **64 traits distintos** en total. El top-15:

Trait	Frecuencia
Item	59
Fortune	29
Ally	25
Blessed	25
Tool	22
Innate	22
Talent	21
Spirit	18
Weapon	17
Melee	16
Tactic	16
Charm	14
Cursed	13
Trick	13
Developed	8

Nos cuadra que **Item** salga primero: es un trait muy general que aparece en todo tipo de cartas. Que **Fortune** esté segundo también tiene sentido, porque los survivors suelen tener mecánicas de “suerte” en el juego. Esta consulta es muy útil para usuarios que quieren hacerse una idea rápida de qué tipo de cartas caracterizan a cada facción.

4.5. II-C: Upgrades con $xp > 0$, excluir mythos y priorizar facción

Esta fue la búsqueda más compleja. El enunciado pide:

- Solo cartas con $xp > 0$ (son mejoras, no cartas base).
- Nunca mostrar cartas de facción **mythos**.
- Permitir filtrar por los puntos de experiencia que le quedan al jugador (xp_max).
- Si hay una facción preferida, sus cartas deben salir primero, pero también se muestran cartas de otras facciones después.
- Paginación de 5 en 5.

Nuestra estrategia final fue usar **dos consultas separadas** cuando hay facción preferida:

1. **Primera query:** el filtro base (xp , traits, exclusión mythos) **más** la restricción de la facción preferida. Esto nos da las cartas “prioritarias”.
2. **Segunda query:** el mismo filtro base pero **excluyendo** la facción preferida. Esto nos da “el resto”.
3. Concatenamos ambos resultados, primero las de la facción preferida y luego las demás, hasta completar el tamaño de página.

Si no hay facción preferida, simplemente se ejecuta una sola query con **FT.SEARCH** y ordenación por xp **ASC**.

```
# Filtro base
xp_filter = "@xp:[(0 5]" # xp>0 y xp<=5
parts = [xp_filter, "-@faction_code:{mythos}"]
# Si hay traits, anadir OR de traits
parts.append("@traits:{spell|item}")
base_query = " ".join(parts)

# 1) Cartas de la faccion preferida
preferred_query = f"{base_query} @faction_code:{{mystic}}"
# 2) Cartas de otras facciones (rellenar)
other_query = f"{base_query} -@faction_code:{{mystic}}"
```

Este enfoque de dos queries tiene la ventaja de ser muy controlable: sabemos exactamente cuántas cartas hay de la facción preferida y cuántas del resto, y la paginación se maneja sin ambigüedades. Además, al ser dos **FT.SEARCH** simples, el parseo es directo y evitamos problemas con campos vacíos que nos daban otros enfoques más complejos con **FT.AGGREGATE**.

4.6. Cómo verificamos que las búsquedas son correctas

Implementamos un sistema de comprobación por asserts. La idea es ejecutar cada búsqueda y validar los resultados programáticamente:

- **A (AND/OR):** con la función **check_A**, recorremos cada carta devuelta y comprobamos que en modo AND todas las facciones pedidas están presentes en **faction_code**, y en modo OR al menos una lo está.

- **B (traits):** comparamos las frecuencias que devuelve el agregado con un conteo manual en Python sobre el DataFrame filtrado por facción. Si coinciden, la lógica de split + groupby es correcta.
- **C (upgrades):** verificamos tres cosas en cada resultado: que `xp > 0`, que `xp <= xp_max` si se aplica, y que `mythos` no aparezca en `faction_code`.

5. Objetivo III: Recomendador de cartas

Este objetivo fue el que más nos gustó, porque se nota muy bien la diferencia entre “parecido por etiquetas”, “parecido por palabras” y “parecido por significado”.

5.1. Embeddings y representación vectorial

Generamos embeddings combinando nombre y texto de cada carta (`name + text`) con el modelo `sentence-transformers/all-MiniLM-L6-v2`. El resultado es una matriz de **(5346, 384)**, es decir, un vector de dimensión 384 por carta. Para almacenarlos en Redis, los convertimos a un blob binario de `float32` con `.tobytes()`.

Además implementamos una función `asegurar_embeddings` que recorre todos los hashes y solo genera el embedding de aquellas cartas que aún no lo tienen. Así evitamos recalcular todo cada vez que ejecutamos el notebook.

5.2. Índice para recomendación

Creamos un segundo índice (`idx_cards`) que soporta las tres alternativas:

- **A) Metadatos:** `faction_code` y `traits` como TAG con separador `|`.
- **B) Full-text:** `name` y `text` como TEXT.
- **C) Vector search:** campo `embedding` como VECTOR con algoritmo **HNSW** y métrica **COSINE**.

```
r.execute_command(
    "FT.CREATE", "idx_cards", "ON", "HASH",
    "PREFIX", "1", "card:", "SCHEMA",
    "faction_code", "TAG", "SEPARATOR", "|",
    "traits", "TAG", "SEPARATOR", "|",
    "name", "TEXT",
    "text", "TEXT",
    "embedding", "VECTOR", "HNSW", "6",
    "TYPE", "FLOAT32",
    "DIM", "384",
    "DISTANCE_METRIC", "COSINE"
)
```

5.3. Alternativa A: Recomendación por metadatos

La idea es sencilla: dada una carta base, buscar otras que compartan su facción y tengan todos sus traits. Si la carta base es multifacción, buscamos cartas que compartan al menos una de esas facciones.

La query queda del estilo: `@faction_code:{guardian} @traits:{Item} @traits:{Weapon} @traits:{Firearm}`

Excluimos la propia carta con `-@code:{1029}` para que no se recomiende a sí misma.

5.4. Alternativa B: Recomendación full-text

Aquí extraemos las palabras más relevantes del texto de la carta base y hacemos una búsqueda full-text con ellas. Un detalle importante de nuestra implementación: usamos **nlTK** para eliminar stopwords en inglés y nos quedamos solo con palabras de 4 o más caracteres (hasta 8 palabras). Además, decidimos buscar **solo en el campo text** y no en **name**, porque el nombre suele ser demasiado específico y no aporta a la similitud funcional entre cartas.

```
stop_words = set(stopwords.words('english'))
s = (name + " " + text).lower()
s = re.sub(r"[^a-z0-9\s]", " ", s)
words = [w for w in s.split() if len(w) >= 4][:8]
words = [w for w in words if w not in stop_words]
or_part = "|".join(words)
query = f"(@text:({or_part})) -@code:{{{code}}}"
```

5.5. Alternativa C: Recomendación semántica (embeddings + KNN)

Para la búsqueda semántica generamos el embedding de la carta base y hacemos una consulta KNN sobre el índice vectorial. Usamos la API de alto nivel de **redis-py**:

```
q = Query(f"*=>[KNN {k+1} @embedding $vec AS score]")
    .sort_by("score")
    .return_fields("code","name","score","faction_code",
                  "traits","text","image_url")
    .dialect(2)
res = r.ft(index_name).search(q,
    query_params={"vec": vec.tobytes()})
```

Pedimos $k + 1$ vecinos porque uno de ellos será la propia carta base, que luego descartamos. El score devuelto es la distancia coseno (menor = más similar).

5.6. Prueba comparativa con la carta 1029 (Shotgun)

Ejecutamos las tres alternativas usando la carta `code = 1029` (*Shotgun*). Esta carta es de facción **guardian** y tiene traits **Item|Weapon|Firearm**.

5.6.1. Resultados A: Metadatos

- **Candidatos totales: 15**
- Top-5 devuelto: Springfield M1903, Lightning Gun, .32 Colt, Remington Model 1858 y .45 Thompson.

Todas son armas de fuego de la facción **guardian**. Los resultados son coherentes, pero el conjunto es pequeño (15) porque exigimos coincidencia en facción **y** en todos los traits simultáneamente. Además, el orden no refleja grado de similitud: simplemente salen en el orden interno de Redis.

5.6.2. Resultados B: Full-text

- **Candidatos totales: 2304**
- Top-5 devuelto: Cookie's Custom .32, James "Cookie" Fredericks, Old Shotgun, y otras cartas que comparten vocabulario de combate.

El contraste con A es brutal: de 15 a 2304 candidatos. El motivo es que las palabras clave que extrae de *Shotgun* (*ammo, spend, fight, combat, attack, damage...*) son extremadamente frecuentes en el juego —casi la mitad de las cartas mencionan alguna—. Los primeros resultados son razonables (Old Shotgun aparece en el top, Cookie’s Custom .32 comparte mecánicas de munición), pero hay bastante ruido: cartas que mencionan “fight” o “damage” sin tener nada que ver con una escopeta.

5.6.3. Resultados C: Semántica (embeddings + KNN)

- **Devueltos: 5 (KNN)**
- Top-5 con score de distancia coseno:

#	Carta	Score
1	Old Shotgun	0.1506
2	Sawed-Off Shotgun	0.2253
3	Remington Model 1858 (v1)	0.3365
4	Remington Model 1858 (v2)	0.3394
5	Old Hunting Rifle	0.3866

Este resultado es el que más “sentido humano” tiene. Si pensamos en qué se parece a una escopeta, la respuesta obvia son otras escopetas y rifles, y eso es exactamente lo que devuelve. Además, el score da un ranking continuo: Old Shotgun es clarísimamente la más cercana (0.15), y la similitud va decayendo de forma gradual.

6. Discusión y análisis comparativo

6.1. ¿Qué aporta cada enfoque?

A (metadatos) se apoya en información estructurada: facción y traits. Las recomendaciones son muy justificables (“te lo recomiendo porque es de tu facción y tiene los mismos traits”), pero el conjunto es limitado (15 candidatos) y el orden no aporta información real sobre similitud. Es la opción más sencilla y predecible.

B (full-text) cambia radicalmente el criterio: aquí la similitud la determinan las palabras compartidas en el texto. Esto genera un conjunto enorme de candidatos (2304) porque palabras como “fight”, “combat” o “damage” aparecen en casi la mitad de las cartas. Los primeros resultados son razonables, pero el método no distingue bien entre una carta que realmente funciona como una escopeta y otra que simplemente menciona las mismas palabras. Haber añadido la eliminación de stopwords con nltk y buscar solo en `text` (sin el nombre) mejoró la calidad respecto a versiones anteriores.

C (semántica) captura la similitud a nivel de significado. Los 5 resultados son todos armas de fuego con mecánicas prácticamente idénticas a Shotgun (gastar munición, atacar, daño variable), y el ranking por distancia coseno es cuantitativamente coherente. Old Shotgun sale primera con un score muy bajo (0.15, muy cercano), y luego la similitud va decayendo gradualmente. Es el enfoque que mejor responde a la pregunta “¿qué cartas se parecen realmente a esta?”.

6.2. Coste vs calidad

Si lo pensamos como ingenieros:

- **A** es barato computacionalmente y no necesita nada más allá de los tags ya existentes. Robusto y explicable, pero limitado.

- **B** es intermedio: usa el índice full-text que ya tenemos. Barato en recursos, pero la calidad depende mucho de que el vocabulario de las cartas se solape, y en este juego hay demasiadas palabras comunes de combate.
- **C** da la mejor calidad con diferencia, pero exige precomputar embeddings para las 5346 cartas, almacenarlos en Redis ($384 \text{ floats} \times 4 \text{ bytes} \approx 1.5 \text{ KB}$ por carta) y mantener un índice HNSW. Merece la pena si la calidad de recomendación importa.

7. Conclusiones

En esta práctica implementamos un flujo completo de **caché + búsqueda avanzada + recomendación** sobre Redis.

La caché con Hashes nos dio un CRUD limpio y eficiente por `code`. Con RediSearch resolvimos búsquedas realistas del portal: facciones AND/OR para cartas multifacción, traits más frecuentes por facción con `FT.AGGREGATE`, y upgrades con filtros de experiencia y priorización de facción. Este último caso lo resolvimos con una estrategia de dos queries que resultó ser más robusta y fácil de depurar que un solo aggregate complejo.

En recomendación, la comparación con **1029 (Shotgun)** dejó claras las fortalezas y debilidades de cada enfoque:

- A devuelve 15 candidatos, todos coherentes pero sin orden significativo.
- B encuentra 2304 candidatos con bastante ruido léxico, aunque los primeros resultados son razonables.
- C produce el ranking más preciso: Old Shotgun con score 0.15, seguida de Sawed-Off Shotgun con 0.23, y luego rifles con similitud decreciente. Es el resultado que un jugador consideraría más “natural”.

Nota final. De las prácticas que hemos hecho en la carrera, esta nos pareció de las más completas porque junta bases de datos, extracción de información y un toque de machine learning aplicado. Y cuando ves que el recomendador semántico te devuelve *Old Shotgun* como primera recomendación para *Shotgun* ves que esto funciona de verdad.